



# Class Notes: Session 5 - GitHub as a Project Repository

## Introduction to GitHub

GitHub is a cloud-based platform that hosts Git repositories, facilitating collaboration, version control, and project management. It extends Git's capabilities with features like pull requests, issue tracking, and project boards, making it a central hub for software development teams.

- **Key Features:**

- Cloud-hosted Git repositories (public, private, or internal).
- Collaboration tools: pull requests, code reviews, and team management.
- Integration with CI/CD pipelines, wikis, and GitHub Pages.
- User-friendly web interface and desktop application.

- **Use Case:**

A distributed team uses GitHub to store and manage their project's codebase, track issues, and review contributions from multiple developers.

- **Scenario:**

A startup uses GitHub to host a repository for their mobile app, allowing developers in different time zones to collaborate, track bugs, and manage releases.

- **Source:** *Version Control with Git* by Jon Loeliger and Matthew McCullough.

## GitHub Web Interface

The GitHub web interface provides a browser-based environment to manage repositories without command-line tools. It supports creating/editing files, managing branches, viewing history, and collaborating via issues and pull requests.

- **Key Functionalities:**

- View and edit files directly in the browser.
- Create and manage branches, pull requests, and issues.
- Access repository settings, collaborators, and project boards.
- Visualize commit history and file diffs.

- **Example:**

A developer navigates to their repository on [GitHub.com](https://github.com), edits `README.md` in the web interface, and commits the change with the message “Updated project description.”

- **Scenario:**

A project manager uses the GitHub web interface to review a team’s progress by checking open issues and pull requests, leaving comments without using Git commands.

- **Source:** *Building Tools with GitHub* by Rosemie Callewaert and Johan Revilla.

## Differences Between Git and GitHub

Git and GitHub are complementary but distinct tools in version control workflows.

| Aspect         | Git                                                              | GitHub                                                                   |
|----------------|------------------------------------------------------------------|--------------------------------------------------------------------------|
| Definition     | Distributed version control system for tracking changes locally. | Cloud platform for hosting Git repositories and collaboration.           |
| Operation      | Runs locally, supports offline work.                             | Requires internet for repository access and collaboration.               |
| Features       | Core version control: commits, branches, merges.                 | Adds collaboration: pull requests, issues, project boards, GitHub Pages. |
| Access Control | No built-in user authentication or permissions.                  | User authentication, team permissions, and visibility settings.          |
| Interface      | Command-line or third-party GUI tools (e.g., SourceTree).        | Web interface, GitHub Desktop, and CLI integration.                      |

- **Use Case:**

A solo developer uses Git locally for a personal project, then pushes it to GitHub to share with collaborators or make it publicly available.

- **Scenario:**

A freelancer uses Git to manage a project offline during travel, then syncs it to GitHub to allow a client to review the code.

- **Source:** *The Ultimate Beginners Guide Git* by Adney Ainley.

# Installing GitHub Desktop

GitHub Desktop is a graphical user interface (GUI) for Git, simplifying repository management, committing, and syncing with GitHub.

- **Installation Steps** (Windows/Mac):

- i. Download GitHub Desktop from [desktop.github.com](https://desktop.github.com).
- ii. Run the installer and follow the setup wizard.
- iii. Sign in with your GitHub account.
- iv. Configure Git settings (name, email) during initial setup.

- **Example:**

A student installs GitHub Desktop, signs in, and clones a repository to their Mac, using the GUI to view changes and commit updates.

- **Scenario:**

A beginner developer uses GitHub Desktop to avoid complex Git commands, committing and pushing changes to a team repository with ease.

- **Source:** *Version Control with Git* by Jon Loeliger and Matthew McCullough.

## Creating a Repository in GitHub

A GitHub repository is a storage space for project files, version history, and collaboration tools. Repositories can be public, private, or internal (for organizations).

- **Steps to Create a Repository:**

- i. Log in to [GitHub.com](https://github.com).
- ii. Click the “+” icon (top-right) and select “New repository.”
- iii. Enter a name (e.g., `my-app`).
- iv. Add a description (optional).
- v. Set visibility (public/private).
- vi. Initialize with a README, `.gitignore`, or license (optional).
- vii. Click “Create repository.”

- **Example:**

A developer creates a repository named `blog-site`, initializes it with a README and a `.gitignore` for Node.js, and sets it to public.

- **Scenario:**

A team creates a private repository for a client's e-commerce project to securely store code and documentation.

- **Source:** *Git* by Ryan Hodson.

## Branching in GitHub

Branching in GitHub allows parallel development, enabling developers to work on features or fixes without affecting the main branch (typically `main`).

- **Steps to Create a Branch:**

- i. Navigate to a repository on GitHub.
- ii. Click the branch dropdown (default: `main`).
- iii. Enter a branch name (e.g., `feature-signup`).
- iv. Click "Create branch."
- v. Alternatively, via Git:

```
git checkout -b feature-signup; git push origin feature-signup.
```

- **Example:**

A developer creates a branch `add-footer` on GitHub, adds a file `footer.html`, and commits it via the web interface.

- **Scenario:**

A team creates branches `feature-cart` and `bugfix-payment` to work on different aspects of an e-commerce app concurrently.

- **Source:** *Version Control with Git* by Jon Loeliger and Matthew McCullough.

## Viewing Branches

GitHub provides tools to view all branches in a repository, including their commit history and status.

- **Steps to View Branches:**

- i. Navigate to a repository on GitHub.
- ii. Click the "Branches" tab or select "View all branches" from the branch dropdown.
- iii. Review active branches, their commits, and whether they're ahead/behind `main`.

- **Example:**

A project lead checks the "Branches" tab and sees `feature-signup` is two commits ahead

of `main`.

- **Scenario:**

A team lead monitors branches to ensure developers are working on assigned tasks and identifies branches ready for merging.

- **Source:** *Building Tools with GitHub* by Rosemie Callewaert and Johan Revilla.

## Deleting Branches

Branches can be deleted after merging or if no longer needed to keep the repository organized.

- **Steps to Delete a Branch:**

- i. Go to the “Branches” tab in the repository.
- ii. Locate the branch (e.g., `feature-signup`).
- iii. Click the trash icon to delete.
- iv. Alternatively, via Git: `git push origin --delete feature-signup`.

- **Example:**

After merging `add-footer` into `main`, a developer deletes it:

```
git push origin --delete add-footer
```

- **Scenario:**

A team deletes obsolete branches like `old-ui` after a redesign to maintain a clean repository.

- **Source:** *The Ultimate Beginners Guide Git* by Adney Ainley.

## Other Operations

GitHub supports additional operations to enhance collaboration and project management:

- **Pull Requests:** Propose, review, and merge changes from one branch to another.
- **Issues:** Track bugs, tasks, or feature requests.
- **Wikis:** Document project details and guidelines.
- **Actions:** Automate workflows (e.g., CI/CD pipelines).
- **Projects:** Organize tasks using Kanban-style boards.
- **Example:**

A developer opens a pull request for `feature-signup` , linking it to issue #10 about user registration, and assigns it for team review.

- **Scenario:**

A team uses GitHub Issues to track reported bugs and a project board to prioritize tasks for the next sprint.

- **Source:** *Version Control with Git* by Jon Loeliger and Matthew McCullough.

## Committing in GitHub

Commits in GitHub can be made via the web interface, command line, or GitHub Desktop, saving changes to the repository with descriptive messages.

- **Steps to Commit via GitHub Desktop:**

- i. Open the repository in GitHub Desktop.
- ii. Make changes to files locally.
- iii. View changes in the “Changes” tab.
- iv. Write a commit message (e.g., “Added signup page”).
- v. Click “Commit to .”
- vi. Push to GitHub: “Push origin.”

- **Example:**

A developer edits `style.css` , commits via GitHub Desktop with “Updated styles,” and pushes to the `main` branch.

- **Scenario:**

A team member commits a bug fix to a branch via the GitHub web interface, allowing quick updates without local setup.

- **Source:** *Git* by Ryan Hodson.

## Classwork

1. **Create a Repository:**

- Create a new public repository on GitHub named `team-project` .
- Initialize it with a README and a `.gitignore` for Python.
- Clone it to your local machine using GitHub Desktop.

2. **Branching and Committing:**

- Create a branch `feature-login` in GitHub Desktop or the web interface.

- Add a file `login.py` with a simple function.
- Commit with the message “Added login function” and push to GitHub.

### 3. **Manage Branches:**

- View all branches in the repository via the GitHub web interface.
- Delete the `feature-login` branch after merging it into `main` (simulate merging via the web interface).

## Session Test

### 1. **Multiple Choice:**

- What is the primary function of GitHub compared to Git?
  - A) Local version control
  - B) Hosting and collaboration platform
  - C) Code compilation
  - D) File encryption
  - **Answer: B**

### 2. **Short Answer:**

- Explain how the GitHub web interface simplifies repository management for non-technical users.

### 3. **Practical:**

- Create a repository named `test-repo`, add a file `index.html` via the GitHub web interface, commit it, create a branch `feature-header`, and push a change to it.

### 4. **True/False:**

- GitHub Desktop requires command-line knowledge to commit changes. (False)

### 5. **Scenario-Based:**

- Describe how a team can use GitHub to create a repository, manage branches for a new feature, and commit changes using both the web interface and GitHub Desktop.